

Python ALA 2 = Variables and Constants in Python

1. What is a variable?

A **variable** is a name that refers to a **memory location** where a value is stored.

In simple terms:

Variable = container that holds a value. `geeksforgeeks+1`

Syntax:

python

`variable_name = value`

Examples:

python

`x = 10` *# integer*

`name = "Irfan"` *# string*

`price = 99.99` *# float*

Here:

- x stores an integer (int)
- name stores a string (str)
- price stores a floating-point number (float) [w3schools](#)

2. How Python handles variables

Python is **dynamically typed**, which means:

- You **do not** write `int x`; or `str name`;
- Python **automatically** decides the type from the value you assign. `realpython+1`

Example:

python

`x = 10` *# now x is int*

`x = "Hello"` *# now x is str*

The **same variable** can store **different types** at different times. `geeksforgeeks+1`

3. Memory concept (interview-style)

When you write:

```
python
```

```
a = 10
```

Python does the following:

1. Creates an **object** with value 10 in memory.
2. Makes the variable **a** **refer to** (point to) that object. [geeksforgeeks](https://www.geeksforgeeks.org/python-variables/)

So:

- A variable **stores a reference**, not the raw value.
- Two variables can refer to the **same object** until you reassign one. [geeksforgeeks](https://www.geeksforgeeks.org/python-variables/)

4. Rules for naming variables

You must follow these naming rules in Python: [w3schools](https://www.w3schools.com/python/python_variables.asp)

Rule	Valid example	Invalid example
Starts with a letter or _	name, _age ✓	1name ✗
No special symbols (except _)	student_name ✓	name@ ✗
Case-sensitive	Age ≠ age ✓	Age vs age
Cannot use keywords	if_, for_ ✓	if, for ✗

Also, follow **PEP 8** style:

- Use snake_case for variables: student_name = "Irfan" [realpython](https://realpython.com/python-pep8/)

5. Types of variables in Python

- **Local variable** – Defined inside a function and only visible within that function.

```
python
```

```
def demo():
```

```
    x = 5 # local variable
```

- **Global variable** – Defined outside functions and can be read from anywhere.

```
python
```

```
x = 10 # global
```

```
def show():
```

```
print(x) # prints 10
```

- **Instance variable** – Used in classes and belongs to each object (instance).

python

```
class Student:
```

```
    def __init__(self):
```

```
        self.name = "Irfan" # instance variable
```

These distinctions are important for interviews and design-level questions.[realpython+1](#)

6. Multiple assignment

You can assign values in one line:

python

```
a = b = c = 10    # all three get 10
```

```
x, y, z = 1, 2, 3 # tuple unpacking
```

This is just **syntactic sugar** for several assignments.[realpython](#)

7. Type checking

Use the built-in `type()` function:

python

```
x = 10
```

```
print(type(x)) # <class 'int'>
```

You can also check with `isinstance()` for more robust checks.[realpython](#)

8. What is a constant?

A **constant** is a value that **should not change** during execution.

Python does **not** have true constants like `const` in C/C++.

Instead, we **use naming convention**:

python

```
PI = 3.14
```

```
GRAVITY = 9.8
```

```
MAX_LIMIT = 100
```

Important note:

python

PI = 3.14

PI = 10 *# This is allowed but logically wrong*

Technically Python allows reassignment, but by convention we treat UPPER_CASE identifiers as constants.[press.rebus](https://www.press.rebus.com)

9. Variable vs constant

Feature	Variable	Constant
Value	Can change anytime	Should not change
Naming style	snake_case (e.g., age)	UPPER_CASE (e.g., PI)
Example	age = 20	PI = 3.14

This distinction helps other programmers understand your intent.[byjus](https://www.byjus.com)+1

10. Complete program example

```
python
```

```
# Variables
```

```
name = "Irfan"
```

```
age = 20
```

```
salary = 15000.50
```

```
# Constants
```

```
PI = 3.14
```

```
GRAVITY = 9.8
```

```
# Display
```

```
print("Name:", name)
```

```
print("Age:", age)
```

```
print("Salary:", salary)
```

```
print("PI Value:", PI)
```

```
print("Gravity:", GRAVITY)
```

Changing variable

age = 21

print("Updated Age:", age)

Output:

text

Name: Irfan

Age: 20

Salary: 15000.5

PI Value: 3.14

Gravity: 9.8

Updated Age: 21

11. Advanced / interview-style points

- **Mutable vs immutable:**
 - **Immutable:** int, float, str, tuple – their value cannot be changed in place; operations create a new object.toronto+2
 - **Mutable:** list, dict, set – their contents can be changed and the same object remains.unstop+1
- **Naming convention (PEP 8):**
 - Variables → snake_case (student_name).
 - Constants → UPPER_CASE (MAX_AGE).[realpython](#)
- **Dynamic binding:**

python

x = 5

x = "Python" *# same variable, new type, new object*

- **Reference vs value:**

Python variables store **references** to objects; when you assign a = b, both variables refer to the **same object** until you rebind one.[geeksforgeeks](#)